

Alright.

Now we do this **properly end-to-end**, exactly how a **real backend architect** would design it.

Below is a **complete API flow** covering:

- Authentication
- Role-based access
- Citizen flow
- Admin / Coordinator flow
- Voting, severity, map, status proof

This can directly become your **backend blueprint**.

End-to-End API Design

Community Waste Reporting Platform

Base URL:

/api/v1

Authentication APIs

◆ Signup

POST /auth/signup

Request

```
{  
  "name": "Kalpesh",  
  "email": "kalpesh@gmail.com",  
  "phone": "9XXXXXXXXX",  
  "password": "*****",  
  "role": "citizen",  
  "state": "Maharashtra",  
  "district": "Mumbai Suburban"
```

```
}
```

Response

```
{  
  "success": true,  
  "message": "Account created successfully"  
}
```

◆ Login

POST /auth/login

Response

```
{  
  "success": true,  
  "token": "JWT_TOKEN",  
  "user": {  
    "id": "userId",  
    "role": "citizen",  
    "district": "Mumbai Suburban"  
  }  
}
```

◆ Auth Middleware (GLOBAL)

- Verify JWT
 - Attach req.user
 - Role-based checks later
-

2 Citizen APIs

◆ Create Waste Report

POST /reports

🔒 Auth required (Citizen)

Form-Data

- image
- description
- latitude
- longitude
- addressText

Backend Flow

1. Upload image → Cloudinary
2. Save report
3. Set status = untouched
4. severityScore = initial value

Response

```
{  
  "success": true,  
  "message": "Report submitted successfully"  
}
```

◆ Get Home Feed (Reports List)

GET /reports

Query Params

?sort=severity | latest | nearby

Response

```
[  
  {  
    "id": "reportId",  
    "imageUrl": "...",
```

```
"description": "...",
"status": "untouched",
"votesCount": 12,
"severityScore": 34,
"createdAt": "..."
}
]
```

◆ Vote on a Report

POST /reports/:id/vote

 Citizen only

Backend Flow

1. Check if user already voted
2. Insert vote
3. Increment votesCount
4. Recalculate severityScore

Response

```
{
  "success": true,
  "message": "Vote registered"
}
```

◆ Check Status (My Reports)

GET /reports/my

Response

```
[
  {
    "reportId": "...",
```

```
"status": "in_progress",  
"lastUpdated": "..."  
}  
]
```

Map APIs (Citizen + Admin)

◆ Get Map Data (GeoJSON)

GET /map/reports

Response

```
{  
  "type": "FeatureCollection",  
  "features": [  
    {  
      "type": "Feature",  
      "geometry": {  
        "type": "Point",  
        "coordinates": [72.8777, 19.0760]  
      },  
      "properties": {  
        "status": "untouched",  
        "severityScore": 42  
      }  
    }  
  ]  
}
```

Frontend maps:

- status → color

- severity → shade intensity

Admin / Coordinator APIs

◆ Get Reports (District Filtered)

GET /admin/reports

 Admin / Coordinator only

Auto-filter

- Based on req.user.district

◆ Update Report Status (WITH PROOF)

PUT /admin/reports/:id/status

Form-Data

- newStatus (in_progress / resolved)
- proofImage

Backend Flow

1. Upload proof image
2. Save in status_updates
3. Update report status
4. Update timestamp

Response

```
{  
  "success": true,  
  "message": "Status updated successfully"  
}
```

◆ View Status History

GET /admin/reports/:id/history

Response

```
[
  {
    "previousStatus": "untouched",
    "newStatus": "in_progress",
    "proofImageUrl": "...",
    "updatedBy": "Coordinator Name",
    "timestamp": "..."
  }
]
```

5 Severity Recalculation APIs

◆ Internal Severity Update

POST /internal/recalculate-severity/:reportId

Called when:

- Vote added
 - Daily cron job
 - Admin update
-

6 Profile APIs

◆ Get Profile

GET /users/me

◆ Update Profile

PUT /users/me

7 Notifications APIs (Optional)

◆ Get Notifications

GET /notifications

◆ Mark as Read

PUT /notifications/:id/read

8 Error Handling (STANDARDIZED)

```
{  
  "success": false,  
  "errorCode": "UNAUTHORIZED",  
  "message": "Access denied"  
}
```

9 API Flow Summary (Citizen → Resolution)

Citizen reports issue

↓

Other citizens vote

↓

Severity increases

↓

Admin sees high priority on map

↓

Admin updates status with proof



Citizen sees progress & resolution

Why This API Design Is STRONG

You can confidently say:

“We designed a REST-based, role-aware API architecture with geospatial responses, proof-based workflows, and scalable severity analytics.”

This is **industry-ready**.